

DATE:22.09.2025	Implementation of Bubble Sort using Array ADT
EXP NO:12a	

Aim

To implement the Bubble Sort algorithm using an Array Abstract Data Type (ADT) for efficient sorting of elements.

Algorithm

1. **Input Phase:** Accept array elements from the user
2. **Sorting Phase:**
 - Compare adjacent elements repeatedly
 - Swap if they are in wrong order
 - Optimized with early termination if no swaps occur
3. **Output Phase:** Display the sorted array

Program

```
#include <stdio.h>

#define MAX 100
typedef struct {
    int arr[MAX];
    int size;
} ArrayADT;

void inputArray(ArrayADT *a) {
    printf("Enter number of elements (max %d): ", MAX);
    scanf("%d", &a->size);

    printf("Enter %d elements:\n", a->size);
    for (int i = 0; i < a->size; i++) {
        scanf("%d", &a->arr[i]);
    }
}
```

```

void displayArray(ArrayADT *a) {
    if (a->size == 0) {
        printf("Array is empty.\n");
        return;
    }
    printf("Array elements: ");
    for (int i = 0; i < a->size; i++) {
        printf("%d ", a->arr[i]);
    }
    printf("\n");
}

void bubbleSort(ArrayADT *a) {
    int n = a->size;
    int temp;
    for (int i = 0; i < n-1; i++) {
        int swapped = 0;
        for (int j = 0; j < n-1-i; j++) {
            if (a->arr[j] > a->arr[j+1]) {
                // Swap elements
                temp = a->arr[j];
                a->arr[j] = a->arr[j+1];
                a->arr[j+1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0)
            break;
    }
    printf("Array sorted using Bubble Sort.\n");
}

int main() {
    ArrayADT array;
    array.size = 0;
    int choice;

    do {
        printf("\n--- Bubble Sort Menu ---\n");
        printf("1. Input array\n");
        printf("2. Display array\n");
        printf("3. Bubble sort\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
    }

```

```
switch(choice) {  
    case 1:  
        inputArray(&array);  
        break;  
  
    case 2:  
        displayArray(&array);  
        break;  
  
    case 3:  
        if (array.size == 0) {  
            printf("Array is empty. Please input elements first.\n");  
        } else {  
            bubbleSort(&array);  
        }  
        break;  
  
    case 0:  
        printf("Exiting program.\n");  
        break;  
  
    default:  
        printf("Invalid choice! Please try again.\n");  
}  
} while (choice != 0);  
  
return 0;  
}
```

Output

```
C:\Users\ADMIN-IP\Documen X + v

--- Bubble Sort Menu ---
1. Input array
2. Display array
3. Bubble sort
0. Exit
Enter your choice: 1
Enter number of elements (max 100): 5
Enter 5 elements:
1 2 3 4 5

--- Bubble Sort Menu ---
1. Input array
2. Display array
3. Bubble sort
0. Exit
Enter your choice: 2
Array elements: 1 2 3 4 5

--- Bubble Sort Menu ---
1. Input array
2. Display array
3. Bubble sort
0. Exit
Enter your choice: 3
Array sorted using Bubble Sort.

--- Bubble Sort Menu ---
1. Input array
2. Display array
3. Bubble sort
0. Exit
Enter your choice: 2
Array elements: 1 2 3 4 5
```

```
--- Bubble Sort Menu ---
```

```
1. Input array
```

```
2. Display array
```

```
3. Bubble sort
```

```
0. Exit
```

```
Enter your choice: 0
```

```
Exiting program.
```

```
-----  
Process exited after 15.57 seconds with return value 0
```

```
Press any key to continue . . .
```

Result

The program successfully implements Bubble Sort using Array ADT, demonstrating efficient sorting with $O(n^2)$ time complexity in worst-case and $O(n)$ in best-case (when optimized). The Array ADT provides structured data management while Bubble Sort effectively arranges elements in ascending order through adjacent comparisons and swaps.

DATE: 22.09.2025	Implementation of Insertion Sort using Array ADT
EXP NO: 12b	

Aim

To implement the Insertion Sort algorithm using an Array Abstract Data Type (ADT) for efficient sorting of elements by building a sorted portion incrementally.

Algorithm

1. **Input Phase:** Accept array elements from the user
2. **Sorting Phase:**
 - Start from the second element (index 1)
 - Compare with elements in the sorted portion
 - Shift elements greater than key to the right
 - Insert key in correct position
3. **Output Phase:** Display the sorted array

Program

```
#include <stdio.h>

#define MAX 100
typedef struct {
    int arr[MAX];
    int size;
} ArrayADT;

void inputArray(ArrayADT *a) {
    printf("Enter number of elements (max %d): ", MAX);
    scanf("%d", &a->size);

    printf("Enter %d elements:\n", a->size);
    for (int i = 0; i < a->size; i++) {
        scanf("%d", &a->arr[i]);
    }
}
```

```

}
void displayArray(ArrayADT *a) {
    if (a->size == 0) {
        printf("Array is empty.\n");
        return;
    }
    printf("Array elements: ");
    for (int i = 0; i < a->size; i++) {
        printf("%d ", a->arr[i]);
    }
    printf("\n");
}
void insertionSort(ArrayADT *a) {
    int i, key, j;
    for (i = 1; i < a->size; i++) {
        key = a->arr[i];
        j = i - 1;

        while (j >= 0 && a->arr[j] > key) {
            a->arr[j + 1] = a->arr[j];
            j = j - 1;
        }
        a->arr[j + 1] = key;
    }
    printf("Array sorted using Insertion Sort.\n");
}

int main() {
    ArrayADT array;
    array.size = 0;
    int choice;

    do {
        printf("\n--- Insertion Sort Menu ---\n");
        printf("1. Input array\n");
        printf("2. Display array\n");
        printf("3. Insertion sort\n");
        printf("0. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                inputArray(&array);

```

```
        break;

    case 2:
        displayArray(&array);
        break;

    case 3:
        if (array.size == 0) {
            printf("Array is empty. Please input elements first.\n");
        } else {
            insertionSort(&array);
        }
        break;

    case 0:
        printf("Exiting program.\n");
        break;

    default:
        printf("Invalid choice! Please try again.\n");
    }
} while (choice != 0);

return 0;
}
```


Output

```
--- Insertion Sort Menu ---
1. Input array
2. Display array
3. Insertion sort
0. Exit
Enter your choice: 1
Enter number of elements (max 100): 6
Enter 6 elements:
1 2 3 4 5 6

--- Insertion Sort Menu ---
1. Input array
2. Display array
3. Insertion sort
0. Exit
Enter your choice: 2
Array elements: 1 2 3 4 5 6

--- Insertion Sort Menu ---
1. Input array
2. Display array
3. Insertion sort
0. Exit
Enter your choice: 3
Array sorted using Insertion Sort.

--- Insertion Sort Menu ---
1. Input array
2. Display array
3. Insertion sort
0. Exit
Enter your choice: 2
Array elements: 1 2 3 4 5 6
```

```
--- Insertion Sort Menu ---
```

1. Input array
2. Display array
3. Insertion sort
0. Exit

```
Enter your choice: 0
```

```
Exiting program.
```

```
-----  
Process exited after 50.87 seconds with return value 0
```

```
Press any key to continue . . . |
```

Result

The program successfully implements Insertion Sort using Array ADT, demonstrating efficient sorting with $O(n^2)$ time complexity in worst-case and $O(n)$ in best-case (when array is nearly sorted). The algorithm builds the sorted portion incrementally by

inserting each element in its correct position, making it efficient for small datasets and partially sorted arrays.